

Project Update #1 — Gym Equipment Detection with YOLOv7

Team: Varun Gazala | Mohit Raiyani | Jatinkumar Nabhoya
University of New Haven | Spring 2026

Classes we detect (5 custom — none in COCO): | ID | Name | Description | |---|---|---| | 0 | dumbbell | Free weights for arm/shoulder exercises | | 1 | barbell | Long bar for squats/deadlifts | | 2 | kettlebell | Cast iron ball with handle | | 3 | resistance_band | Elastic band for resistance training | | 4 | pull_up_bar | Overhead bar mounted on a rack |

Step 0 — Install & Clone YOLOv7

```
import os

# Clone YOLOv7 repo if not already done
if not os.path.exists('yolov7'):
    !git clone https://github.com/WongKinYiu/yolov7.git

%cd yolov7

# Install required packages
!pip install -q matplotlib numpy opencv-python Pillow PyYAML scikit-learn torch torchvision torchmetrics

# Download pretrained COCO weights
if not os.path.exists('yolov7.pt'):
    !curl -L -o yolov7.pt
    https://github.com/WongKinYiu/yolov7/releases/download/v0.1/yolov7.pt

print('Done.')

/Users/jatinnabhoya/Desktop/UNH/Semester 3/Deep Learning
/Project/phase 1/yolov7
Done.

%matplotlib inline
import sys, os, random, shutil, glob, yaml
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import cv2
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from pathlib import Path
```

```

from sklearn.model_selection import train_test_split

# YOLOv7 utilities
sys.path.insert(0, '.')
from models.experimental import attempt_load
from utils.general import non_max_suppression, xywh2xyxy
from utils.torch_utils import select_device
from utils.datasets import letterbox
from utils.loss import ComputeLoss

# — Config —————
DEVICE      = select_device('')          # auto: CUDA > MPS > CPU
SEED        = 42
IMG_SIZE    = 640
BATCH_SIZE  = 8
DATASET_DIR = '../dataset'              # root folder for images +
labels
CLASS_NAMES = ['dumbbell', 'barbell', 'kettlebell', 'resistance_band',
'pull_up_bar']
NUM_CLASSES = len(CLASS_NAMES)
# one color per class
COLORS = [(220,50,50), (50,180,50), (50,100,220), (240,180,30),
(150,60,210)]

# Reproducibility
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)

print(f'Device: {DEVICE} | PyTorch: {torch.__version__}')
print(f'Classes: {CLASS_NAMES}')

Device: cpu | PyTorch: 2.10.0
Classes: ['dumbbell', 'barbell', 'kettlebell', 'resistance_band',
'pull_up_bar']

%matplotlib inline

```

Step 1 — Dataset Setup

1a. Create folder structure & data.yaml

```

# Create all required folders
for split in ['train', 'val', 'test', 'all']:
    os.makedirs(f'{DATASET_DIR}/images/{split}', exist_ok=True)
    os.makedirs(f'{DATASET_DIR}/labels/{split}', exist_ok=True)

# Write data.yaml that YOLOv7 uses
cfg = {
    'train': os.path.abspath(f'{DATASET_DIR}/images/train'),

```

```

    'val': os.path.abspath(f'{DATASET_DIR}/images/val'),
    'test': os.path.abspath(f'{DATASET_DIR}/images/test'),
    'nc': NUM_CLASSES,
    'names': CLASS_NAMES
}
with open(f'{DATASET_DIR}/data.yaml', 'w') as f:
    yaml.dump(cfg, f, default_flow_style=False)

print('Folders created and data.yaml saved.')
print(f"\ndata.yaml:")
print(yaml.dump(cfg, default_flow_style=False))

```

Folders created and data.yaml saved.

```

data.yaml:
names:
- dumbbell
- barbell
- kettlebell
- resistance_band
- pull_up_bar
nc: 5
test: /Users/jatinnabhoya/Desktop/UNH/Semester 3/Deep Learning
/Project/phase 1/dataset/images/test
train: /Users/jatinnabhoya/Desktop/UNH/Semester 3/Deep Learning
/Project/phase 1/dataset/images/train
val: /Users/jatinnabhoya/Desktop/UNH/Semester 3/Deep Learning
/Project/phase 1/dataset/images/val

```

1b. Dataset partitioning — 80% train / 10% val / 10% test

Put all your images in dataset/images/all/ and labels in dataset/labels/all/. This cell splits them automatically.

```

def split_dataset(root, seed=42):
    all_imgs = sorted(glob.glob(f'{root}/images/all/*.*'))
    all_imgs = [p for p in all_imgs if
p.lower().endswith(('.jpg', '.png', '.jpeg'))]

    if not all_imgs:
        print('No images found in images/all/ – skipping split.')
        return

    # Get the primary class of each image (first label line) for
    stratification
    primary_class = []
    for img in all_imgs:
        lbl = f'{root}/labels/all/{Path(img).stem}.txt'

```

```

        if os.path.exists(lbl):
            with open(lbl) as f:
                first = f.readline().strip()
                primary_class.append(int(first.split()[0]) if first else
0)
            else:
                primary_class.append(0)

    # Split: 80 / 10 / 10
    train, tmp, _, tmp_c = train_test_split(all_imgs, primary_class,
test_size=0.2, random_state=seed, stratify=primary_class)
    val, test, _, _ = train_test_split(tmp, tmp_c,
test_size=0.5, random_state=seed, stratify=tmp_c)

    # Copy files to their split folders
    for split_name, img_list in [('train', train), ('val', val),
('test', test)]:
        for img in img_list:
            shutil.copy2(img, f'{root}/images/{split_name}/')
            lbl = f'{root}/labels/all/{Path(img).stem}.txt'
            if os.path.exists(lbl):
                shutil.copy2(lbl, f'{root}/labels/{split_name}/')

    print(f'Split complete:')
    print(f'  train : {len(train)} images
({len(train)/len(all_imgs)*100:.0f}%)')
    print(f'  val   : {len(val)}   images
({len(val)/len(all_imgs)*100:.0f}%)')
    print(f'  test  : {len(test)}  images
({len(test)/len(all_imgs)*100:.0f}%)')

split_dataset(DATASET_DIR, seed=SEED)

Split complete:
train : 162 images (80%)
val   : 20   images (10%)
test  : 21   images (10%)

```

1c. Custom Dataset class

Reads images and YOLO-format `.txt` label files. Applies augmentation for training.

```

class GymDataset(Dataset):
    def __init__(self, img_dir, lbl_dir, img_size=640, augment=False):
        self.img_dir = img_dir
        self.lbl_dir = lbl_dir
        self.img_size = img_size
        self.augment = augment

    # Collect all image paths

```

```

self.imgs = sorted([
    p for p in glob.glob(f'{img_dir}/*..*')
    if p.lower().endswith(('.jpg', '.png', '.jpeg'))
])

# Read all label files upfront
self.labels = []
for img in self.imgs:
    lbl_path = f"{lbl_dir}/{Path(img).stem}.txt"
    if os.path.exists(lbl_path):
        rows = []
        with open(lbl_path) as f:
            for line in f:
                vals = line.strip().split()
                if len(vals) == 5:
                    rows.append([int(vals[0])] + [float(v) for
v in vals[1:]])
        self.labels.append(np.array(rows, dtype=np.float32) if
rows else np.zeros((0,5), dtype=np.float32))
    else:
        self.labels.append(np.zeros((0,5), dtype=np.float32))

    print(f'Loaded {len(self.imgs)} images | {sum(len(l) for l in
self.labels)} annotations')

def __len__(self):
    return len(self.imgs)

def __getitem__(self, idx):
    # Load image (BGR -> RGB)
    img = cv2.imread(self.imgs[idx])
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    labels = self.labels[idx].copy()

    # Training augmentation (per YOLOv7 paper bag-of-freebies)
    if self.augment:
        # Horizontal flip
        if random.random() < 0.5:
            img = np.fliplr(img).copy()
            if len(labels): labels[:, 1] = 1.0 - labels[:, 1] #
flip x_center

        # HSV jitter (hue ±0.015, sat ±0.7, val ±0.4)
        # if random.random() < 0.5:
        #     r = np.random.uniform(-1,1,3) * [0.015, 0.7, 0.4] +
1
        #     h, s, v = cv2.split(cv2.cvtColor(img,
cv2.COLOR_RGB2HSV))
        #     lut = lambda x, gain: np.clip(np.arange(256)*gain,
0, 255).astype(np.uint8)

```

```

        # img = cv2.cvtColor(cv2.merge([cv2.LUT(h,
(np.arange(256)*r[0])%180), cv2.LUT(s, lut(s,r[1])), cv2.LUT(v,
lut(v,r[2]))]), cv2.COLOR_HSV2RGB)
        # HSV jitter (hue ±0.015, sat ±0.7, val ±0.4)
        if random.random() < 0.5:
            r = np.random.uniform(-1,1,3) * [0.015, 0.7, 0.4] + 1
            h, s, v = cv2.split(cv2.cvtColor(img,
cv2.COLOR_RGB2HSV))
            lut_hue = ((np.arange(256) * r[0]) %
180).astype(np.uint8)
            lut_sat = np.clip(np.arange(256) * r[1], 0,
255).astype(np.uint8)
            lut_val = np.clip(np.arange(256) * r[2], 0,
255).astype(np.uint8)
            img = cv2.cvtColor(cv2.merge([cv2.LUT(h, lut_hue),
cv2.LUT(s, lut_sat), cv2.LUT(v, lut_val)]), cv2.COLOR_HSV2RGB)

        # Letterbox resize to 640x640 (preserves aspect ratio with
padding)
        img, _, _ = letterbox(img, self.img_size, auto=False,
scaleFill=False)

        # Normalize to [0, 1] and convert HWC -> CHW tensor
        img = torch.from_numpy(img.astype(np.float32) /
255.0).permute(2, 0, 1)

        targets = torch.from_numpy(labels).float() if len(labels) else
torch.zeros((0, 5))
        return img, targets, self.imgs[idx]

print('GymDataset defined.')

GymDataset defined.

def collate_fn(batch):
    """Adds batch index column so YOLOv7 loss knows which image each
box belongs to."""
    imgs, targets, paths = zip(*batch)
    imgs = torch.stack(imgs)
    new_targets = []
    for i, t in enumerate(targets):
        if len(t):
            new_targets.append(torch.cat([torch.full((len(t),1), i),
t], dim=1))
    targets = torch.cat(new_targets) if new_targets else
torch.zeros((0,6))
    return imgs, targets, list(paths)

# Create datasets
print('=== Training set ===')

```

```

train_ds = GymDataset(f'{DATASET_DIR}/images/train',
f'{DATASET_DIR}/labels/train', augment=True)
print('=== Validation set ===')
val_ds = GymDataset(f'{DATASET_DIR}/images/val',
f'{DATASET_DIR}/labels/val')
print('=== Test set ===')
test_ds = GymDataset(f'{DATASET_DIR}/images/test',
f'{DATASET_DIR}/labels/test')

# Create dataloaders (num_workers=0 works on macOS and Colab)
train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE,
shuffle=True, collate_fn=collate_fn, num_workers=0)
val_loader = DataLoader(val_ds, batch_size=BATCH_SIZE,
shuffle=False, collate_fn=collate_fn, num_workers=0)
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE,
shuffle=False, collate_fn=collate_fn, num_workers=0)

print(f'\nTrain: {len(train_ds)} imgs, {len(train_loader)} batches')
print(f'Val: {len(val_ds)} imgs, {len(val_loader)} batches')
print(f'Test: {len(test_ds)} imgs, {len(test_loader)} batches')

=== Training set ===
Loaded 162 images | 247 annotations
=== Validation set ===
Loaded 20 images | 35 annotations
=== Test set ===
Loaded 21 images | 33 annotations

Train: 162 imgs, 21 batches
Val: 20 imgs, 3 batches
Test: 21 imgs, 3 batches

```

1d. Visualize sample training images with ground truth boxes

```

from IPython.display import display

fig, axes = plt.subplots(2, 4, figsize=(18, 9))
fig.suptitle('Sample Training Images with Ground Truth Annotations',
fontsize=14, fontweight='bold')

for i, ax in enumerate(axes.flat):
    if i >= len(train_ds):
        ax.axis('off')
        continue

    img, targets, _ = train_ds[i]
    img_np = img.permute(1,2,0).numpy() # CHW -> HWC
    H, W = img_np.shape[:2]

    ax.imshow(img_np)

```

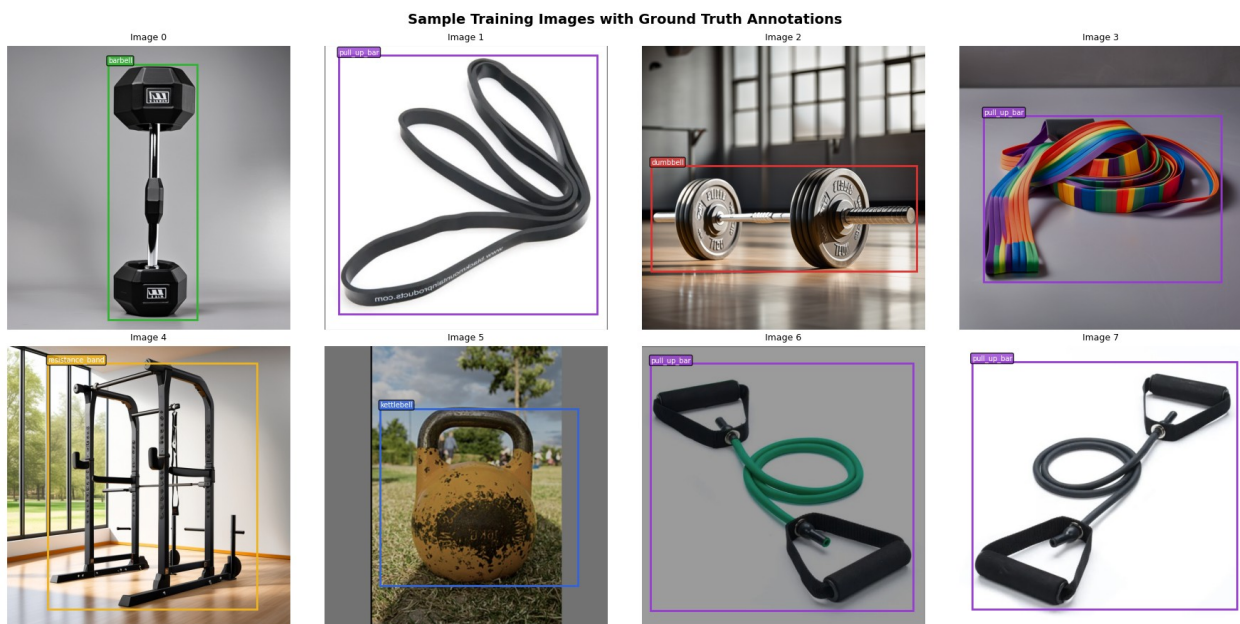
```

ax.set_title(f'Image {i}', fontsize=9)
ax.axis('off')

# Draw each bounding box
for box in targets:
    cls = int(box[0])
    xc, yc, bw, bh = box[1]*W, box[2]*H, box[3]*W, box[4]*H
    x1, y1 = xc - bw/2, yc - bh/2
    color = [c/255 for c in COLORS[cls]]
    ax.add_patch(patches.Rectangle((x1,y1), bw, bh, linewidth=2,
    edgecolor=color, facecolor='none'))
    ax.text(x1, y1-3, CLASS_NAMES[cls], fontsize=7, color='white',
           bbox=dict(boxstyle='round,pad=0.2', facecolor=color,
    alpha=0.8))

plt.tight_layout()
display(fig)
plt.close(fig)

```



Step 2 — Pretrained Model Inference (No Changes)

We load YOLOv7 as-is (trained on 80 COCO classes). Since our gym equipment classes don't exist in COCO, the model will either detect nothing or mislabel objects. This shows why fine-tuning is needed.

```

# Load pretrained YOLOv7 (COCO, 80 classes)
pretrained = attempt_load('yolov7.pt', map_location=DEVICE)
pretrained.eval()

coco_names = pretrained.names

```



```

if isinstance(coco_names, dict):
    coco_names = [coco_names[i] for i in range(len(coco_names))]

total = sum(p.numel() for p in pretrained.parameters())
print(f'YOLOv7 loaded – {total:,} parameters, trained on
{len(coco_names)} COCO classes')

Fusing layers...
RepConv.fuse_repvgg_block
RepConv.fuse_repvgg_block
RepConv.fuse_repvgg_block
YOLOv7 loaded – 36,905,341 parameters, trained on 80 COCO classes

/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/
site-packages/torch/functional.py:505: UserWarning: torch.meshgrid: in
an upcoming release, it will be required to pass the indexing
argument. (Triggered internally at
/Users/runner/work/pytorch/pytorch/pytorch/aten/src/ATen/native/Tensor
Shape.cpp:4383.)
    return _VF.meshgrid(tensors, **kwargs) # type: ignore[attr-defined]

# Run pretrained model on training data – show 1st image from each of
5 batches
MAX_BATCHES = 5

fig, axes = plt.subplots(1, MAX_BATCHES, figsize=(4*MAX_BATCHES, 5))
fig.suptitle('Pretrained COCO Model – 1st Sample per Minibatch (No
Fine-tuning)', fontsize=13, fontweight='bold')

pretrained.eval()
with torch.no_grad():
    for batch_idx, (imgs, targets, paths) in enumerate(train_loader):
        if batch_idx >= MAX_BATCHES:
            break

        preds = non_max_suppression(pretrained(imgs.to(DEVICE))[0],
conf_thres=0.25, iou_thres=0.45)

        # First image in this batch
        img_np = imgs[0].permute(1,2,0).numpy()
        det = preds[0] # detections for image 0

        # Ground truth boxes
        gt = targets[targets[:,0] == 0]
        H, W = img_np.shape[:2]

        ax = axes[batch_idx]
        ax.imshow(img_np)

        # Draw GT (green dashed)
        for box in gt:

```

```

cls = int(box[1])
xc,yc,bw,bh = box[2]*W, box[3]*H, box[4]*W, box[5]*H
x1,y1 = xc-bw/2, yc-bh/2
ax.add_patch(patches.Rectangle((x1,y1),bw,bh, linewidth=2,
edgecolor='lime', facecolor='none', linestyle='--'))
ax.text(x1, y1-3, f'GT:{CLASS_NAMES[cls]}', fontsize=6,
color='lime')

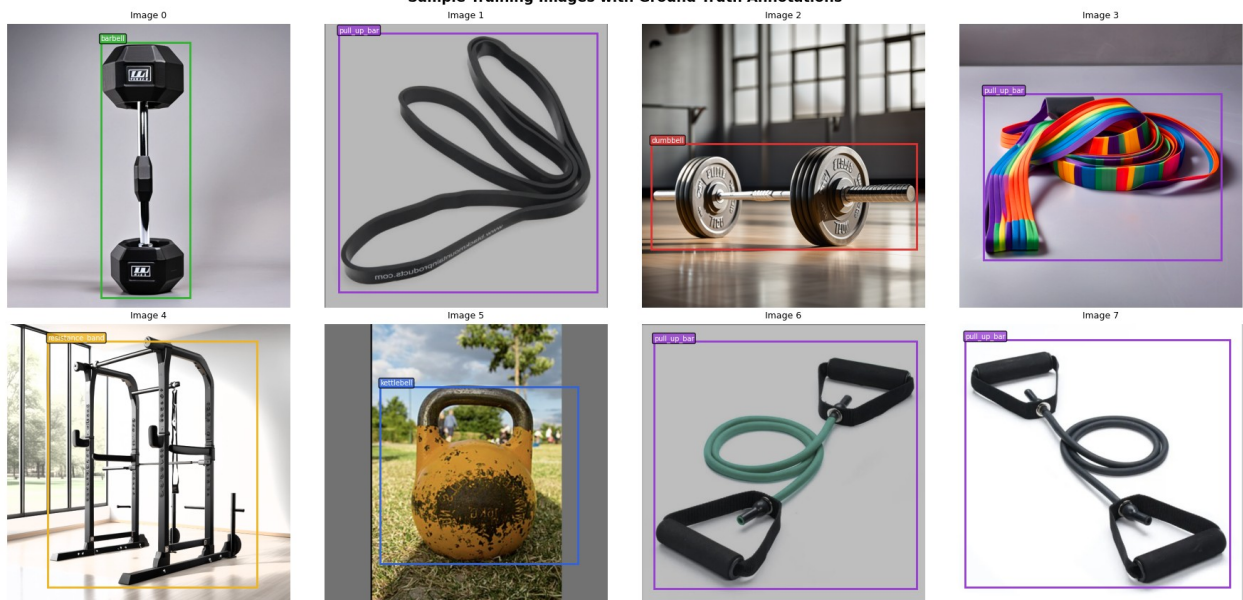
# Draw predictions (red)
n_det = 0
if det is not None and len(det):
    for x1,y1,x2,y2,conf,cls in det.cpu():
        ax.add_patch(patches.Rectangle((x1,y1),x2-x1,y2-y1,
linewidth=2, edgecolor='red', facecolor='none'))
        ax.text(x1, y2+10, f'{coco_names[int(cls)]}:
{conf:.2f}', fontsize=6, color='red')
        n_det += 1

ax.set_title(f'Batch {batch_idx+1}\n{n_det} COCO detections',
fontsize=9)
ax.axis('off')

plt.tight_layout()
plt.show()
print('As expected: model detects COCO objects (or nothing) – NOT our
gym equipment.')

```

Sample Training Images with Ground Truth Annotations





As expected: model detects COCO objects (or nothing) – NOT our gym equipment.

Step 3 — Transfer Learning: Replace Output Layers

YOLOv7's detection head has 3 output scales. Each outputs $(nc + 5) \times 3$ channels per anchor.

We replace the final conv layers: **80 → 5 classes** (255 → 30 output channels).

```
# Load fresh model and replace detection head
model = attempt_load('yolov7.pt', map_location=DEVICE)
detect = model.model[-1] # the Detect module

print(f'Original output layers:')
for i, m in enumerate(detect.m):
    print(f'  Scale {i}: {m}')

# Replace each conv: in_channels stays, out_channels = (5+5)*3 = 30
na = detect.na # 3 anchors per scale
new_nc = NUM_CLASSES # 5
new_no = new_nc + 5 # 10 values per anchor

for i, m in enumerate(detect.m):
    detect.m[i] = nn.Conv2d(m.in_channels, new_no * na, 1).to(DEVICE)

detect.nc = new_nc
detect.no = new_no
model.nc = new_nc
model.names = CLASS_NAMES
```

```

print(f'\nReplaced output layers:')
for i, m in enumerate(detect.m):
    print(f'  Scale {i}: {m}')

backbone_params = sum(p.numel() for p in model.parameters()) -
sum(p.numel() for p in detect.m.parameters())
head_params      = sum(p.numel() for p in detect.m.parameters())
print(f'\nBackbone (pretrained): {backbone_params:,} params')
print(f'Detection head (random init): {head_params:,} params')

Fusing layers...
RepConv.fuse_repvvgg_block
RepConv.fuse_repvvgg_block
RepConv.fuse_repvvgg_block
Original output layers:
  Scale 0: Conv2d(256, 255, kernel_size=(1, 1), stride=(1, 1))
  Scale 1: Conv2d(512, 255, kernel_size=(1, 1), stride=(1, 1))
  Scale 2: Conv2d(1024, 255, kernel_size=(1, 1), stride=(1, 1))

Replaced output layers:
  Scale 0: Conv2d(256, 30, kernel_size=(1, 1), stride=(1, 1))
  Scale 1: Conv2d(512, 30, kernel_size=(1, 1), stride=(1, 1))
  Scale 2: Conv2d(1024, 30, kernel_size=(1, 1), stride=(1, 1))

Backbone (pretrained): 36,447,616 params
Detection head (random init): 53,850 params

```

Step 4 — Baseline Evaluation (No Training)

Since the detection head is randomly initialized, we expect $mAP \approx 0$ and high loss. These numbers are the baseline we'll compare against after training.

4a. mAP on train and validation sets

```

from torchmetrics.detection import MeanAveragePrecision

def compute_map(model, loader):
    metric = MeanAveragePrecision(iou_type='bbox')
    model.eval()

    with torch.no_grad():
        for imgs, targets, _ in loader:
            preds = non_max_suppression(model(imgs.to(DEVICE)))[0],
            conf_thres=0.001, iou_thres=0.45)
            H = W = IMG_SIZE

            for i in range(len(imgs)):
                det = preds[i]
                pred_dict = dict(
                    boxes = det[:, :4].cpu() if det is not None and

```

```

len(det) else torch.zeros((0,4)),
        scores = det[:, 4].cpu() if det is not None and
len(det) else torch.zeros(0),
        labels = det[:, 5].long().cpu() if det is not None
and len(det) else torch.zeros(0,dtype=torch.long)
    )
    gt = targets[targets[:,0] == i]
    if len(gt):
        xyxy = xywh2xyxy(gt[:,2:6].clone())
        xyxy[:, [0,2]] *= W; xyxy[:, [1,3]] *= H
    else:
        xyxy = torch.zeros((0,4))
        gt_dict = dict(boxes=xyxy.cpu(),
labels=gt[:,1].long().cpu() if len(gt) else
torch.zeros(0,dtype=torch.long))
        metric.update([pred_dict], [gt_dict])

    r = metric.compute()
    return r['map_50'].item(), r['map'].item()

print('Evaluating on training set...')
train_map50, train_map = compute_map(model, train_loader)
print('Evaluating on validation set...')
val_map50, val_map = compute_map(model, val_loader)

print(f'\n{"":=<45}')
print(f'Baseline mAP Results (Before Training)')
print(f'{"":=<45}')
print(f'{"Metric":<20} {"Train":>10} {"Val":>10}')
print(f'{"-"*42}')
print(f'{"mAP@0.5":<20} {train_map50:>10.4f} {val_map50:>10.4f}')
print(f'{"mAP@0.5:0.95":<20} {train_map:>10.4f} {val_map:>10.4f}')
print(f'\nExpected: both ≈ 0 because the head has random weights.')

```

```

/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/
site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found.
Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user\_install.html
    from .autonotebook import tqdm as notebook_tqdm

```

Evaluating on training set...

```

/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/
site-packages/torchmetrics/utilities/prints.py:43: UserWarning:
Encountered more than 100 detections in a single image. This means
that certain detections with the lowest scores will be ignored, that
may have an undesirable impact on performance. Please consider
adjusting the `max_detection_threshold` to suit your use case. To
disable this warning, set attribute class

```

```
`warn_on_many_detections=False`, after initializing the metric.  
warnings.warn(*args, **kwargs)
```

Evaluating on validation set...

```
=====
Baseline mAP Results (Before Training)
=====
Metric                                Train      Val
-----
mAP@0.5                               0.0000     0.0000
mAP@0.5:0.95                         0.0000     0.0000
```

Expected: both ≈ 0 because the head has random weights.

4b. Average loss on train and validation sets

```
# Attach hyperparameters (required by ComputeLoss)
model.hyp = {'box':0.05,'obj':0.7,'cls':0.3,'cls_pw':1.0,'obj_pw':1.0,  
            'anchor_t':4.0,'fl_gamma':0.0,'label_smoothing':0.0}
model.gr = 1.0

def compute_avg_loss(model, loader):
    model.train() # ComputeLoss needs train mode
    loss_fn = ComputeLoss(model)
    totals = np.zeros(4) # box, obj, cls, total
    n = 0

    with torch.no_grad():
        for imgs, targets, _ in loader:
            if len(targets) == 0:
                continue
            loss, items = loss_fn(model(imgs.to(DEVICE)),  
targets.to(DEVICE))
            if torch.isfinite(loss):
                items_np = items.cpu().numpy()
                totals[:len(items_np)] += items_np[:4]
                n += 1

    model.eval()
    return (totals / n) if n else totals

print('Computing training loss...')
tr = compute_avg_loss(model, train_loader)
print('Computing validation loss...')
vl = compute_avg_loss(model, val_loader)

print(f'\n{"":=<50}')
print(f'Average Loss Before Training (Baseline)')
```

```

print(f'{"":<50}')
print(f'{"Component":<22} {"Train":>12} {"Val":>12}')
```

```

print(f'{"-"*48}')
```

```

print(f'{"Box loss (CIoU)":<22} {tr[0]:>12.4f} {vl[0]:>12.4f}')
```

```

print(f'{"Objectness loss (BCE)":<22} {tr[1]:>12.4f} {vl[1]:>12.4f}')
```

```

print(f'{"Class loss (BCE)":<22} {tr[2]:>12.4f} {vl[2]:>12.4f}')
```

```

print(f'{"-"*48}')
```

```

print(f'{"Total loss":<22} {tr[3]:>12.4f} {vl[3]:>12.4f}')
```

```

print(f'\nExpected: high values – model has not been trained on our
classes.')
```

Computing training loss...

Computing validation loss...

```

=====
Average Loss Before Training (Baseline)
=====
```

Component	Train	Val
Box loss (CIoU)	0.0748	0.1074
Objectness loss (BCE)	2.2247	2.1872
Class loss (BCE)	0.5439	0.7072
Total loss	2.8434	3.0018

Expected: high values – model has not been trained on our classes.

4c. Visualize 1st sample per minibatch — Ground Truth vs Predictions (No Training)

```

MAX_BATCHES = 5
```

```

fig, axes = plt.subplots(MAX_BATCHES, 2, figsize=(12, 5*MAX_BATCHES))
fig.suptitle('Ground Truth (left) vs Baseline Predictions (right) –
Before Training',
             fontsize=13, fontweight='bold')
```

```

model.eval()
with torch.no_grad():
    for b_idx, (imgs, targets, _) in enumerate(train_loader):
        if b_idx >= MAX_BATCHES:
            break

        preds = non_max_suppression(model(imgs.to(DEVICE)))[0],
        conf_thres=0.01, iou_thres=0.45)

        img_np = imgs[0].permute(1,2,0).numpy()
        H, W = img_np.shape[:2]
        gt = targets[targets[:,0] == 0]
        det = preds[0]
```



```

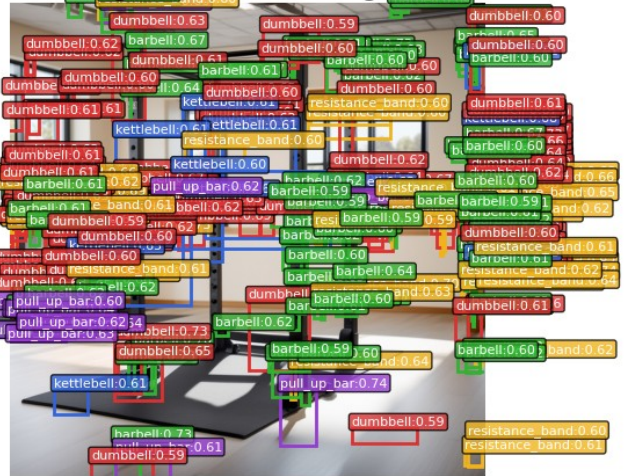
# — Left: Ground Truth —————
ax_gt = axes[b_idx][0]
ax_gt.imshow(img_np)
ax_gt.set_title(f'Batch {b_idx+1} – Ground Truth',
fontsize=10)
ax_gt.axis('off')
for box in gt:
    cls = int(box[1])
    xc,yc,bw,bh = box[2]*W,box[3]*H,box[4]*W,box[5]*H
    x1,y1 = xc-bw/2, yc-bh/2
    color = [c/255 for c in COLORS[cls]]

ax_gt.add_patch(patches.Rectangle((x1,y1),bw,bh,linewidth=2,edgecolor=
color,facecolor='none'))
    ax_gt.text(x1,y1-
4,CLASS_NAMES[cls],fontsize=8,color='white',
bbox=dict(boxstyle='round,pad=0.2',facecolor=color,alpha=0.8))

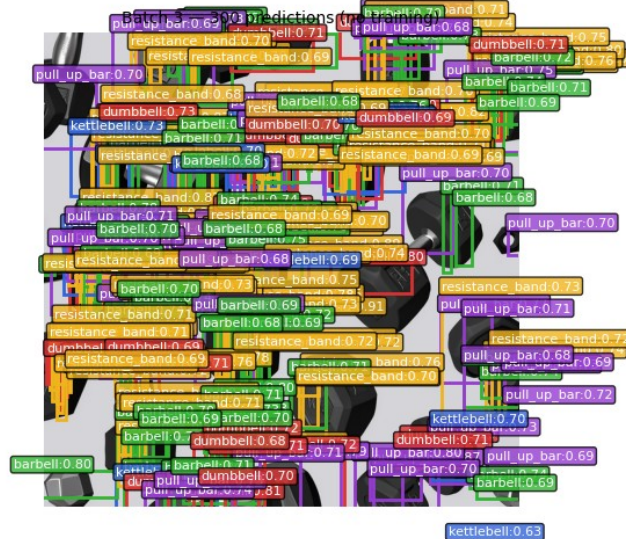
# — Right: Predictions —————
ax_pr = axes[b_idx][1]
ax_pr.imshow(img_np)
ax_pr.axis('off')
n_det = 0
if det is not None and len(det):
    for x1,y1,x2,y2,conf,cls in det.cpu():
        cls = int(cls)
        color = [c/255 for c in COLORS[cls % NUM_CLASSES]]
        ax_pr.add_patch(patches.Rectangle((x1,y1),x2-x1,y2-
y1,linewidth=2,edgecolor=color,facecolor='none'))
        ax_pr.text(x1,y1-4,f'{CLASS_NAMES[cls]}:
{conf:.2f}',fontsize=8,color='white',
bbox=dict(boxstyle='round,pad=0.2',facecolor=color,alpha=0.8))
        n_det += 1
    ax_pr.set_title(f'Batch {b_idx+1} – {n_det} predictions (no
training)', fontsize=10)

plt.tight_layout()
plt.show()
print('Expected: very few or random predictions – head weights are
random.')

```

A collection of dumbbells of various weights and orientations, with several instances highlighted by green bounding boxes and labeled 'barbell'. The labels are placed above each bounding box. The dumbbells are scattered across the frame, with some showing weight markings like '10', '15', '20', '25', and '30' on their hexagonal heads.



Expected: very few or random predictions – head weights are random.

Summary

```
print('=' * 55)
print('PROJECT UPDATE #1 – RESULTS SUMMARY')
print('=' * 55)
print(f'Classes   : {CLASS_NAMES}')
print(f'Dataset    : {len(train_ds)} train / {len(val_ds)} val /
{len(test_ds)} test images')
print(f'Split      : 80% / 10% / 10% (seed={SEED})')
print(f'Batch      : {BATCH_SIZE} | Input: {IMG_SIZE}x{IMG_SIZE}')
print()
print('Baseline (no training):')
print(f' mAP@0.5    – train: {train_map50:.4f} val:
{val_map50:.4f}')
print(f' mAP@0.5:95 – train: {train_map:.4f} val: {val_map:.4f}')
print(f' Total loss – train: {tr[3]:.4f} val: {vl[3]:.4f}')
print()
print('Next: fine-tune the model and compare against these numbers.')
print('=' * 55)
```

```
=====
PROJECT UPDATE #1 – RESULTS SUMMARY
=====
Classes   : ['dumbbell', 'barbell', 'kettlebell', 'resistance_band',
'pull_up_bar']
Dataset    : 162 train / 20 val / 21 test images
Split      : 80% / 10% / 10% (seed=42)
Batch      : 8 | Input: 640x640

Baseline (no training):
  mAP@0.5    – train: 0.0000 val: 0.0000
  mAP@0.5:95 – train: 0.0000 val: 0.0000
  Total loss – train: 2.8434 val: 3.0018
```

```
Next: fine-tune the model and compare against these numbers.
=====
```